Melody Gatan 030269328
Andrea Camacho 030037876
CECS 327 sect 01

P2P Project Report

Video link: https://youtu.be/8o7xPatZQYE?si=arev-Al4_dFrJhcb

System Design:
This project implements a peer-to-peer network with 50+ nodes using Docker. The **bootstrap node** (bootstrap.py) runs on port 5000 and maintains a central registry of peer URLs using Flask and a thread-safe set with **threading.Lock()**. New peers automatically register via POST **/register** when they start up.

The **peer nodes** (node.py) each have a unique UUID and run Flask servers with four endpoints: / (health check), /peers (list known peers), /message (receive messages), and /broadcast (send to all peers). A background thread runs peer_discovery_loop() which registers with the bootstrap after 5 seconds, discovers other peers, then updates the peer list every 30 seconds.

Direct P2P Communication: After discovering peers from the bootstrap, nodes communicate directly without involving the bootstrap. The /broadcast endpoint sends HTTP POST requests directly to each peer's /message endpoint using the discovered peer URLs.

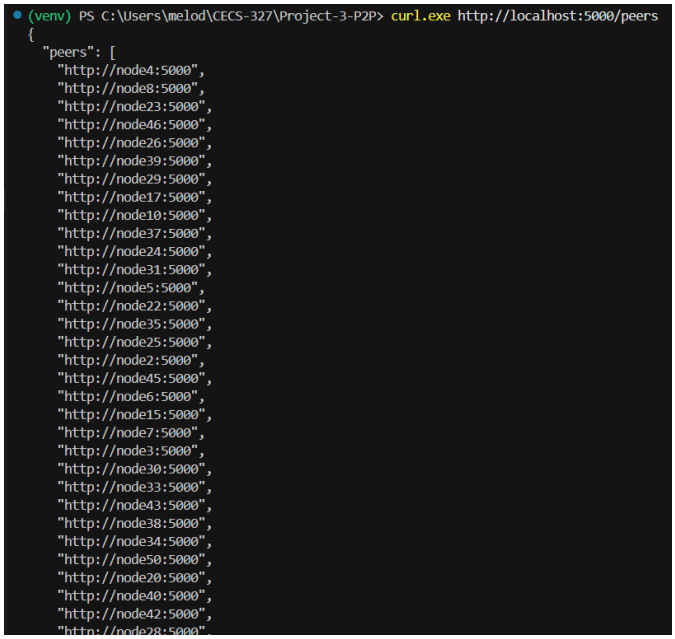
Docker Compose orchestrates 51 containers (1 bootstrap + 50 nodes) on a bridge network. The **generate_compose.py** script uses **PyYAML** to dynamically create **docker-compose-large.yml** with all 50 node definitions and proper port mappings (5001-5050).
How Peer Discovery Works:

1. Node starts and waits 5 seconds
2. Node registers itself with bootstrap: **POST http://bootstrap:5000/register**
3. Node requests peer list: **GET http://bootstrap:5000/peers**
4. Node stores peer URLs locally (excluding itself)
5. Node updates peer list every 30 seconds automatically

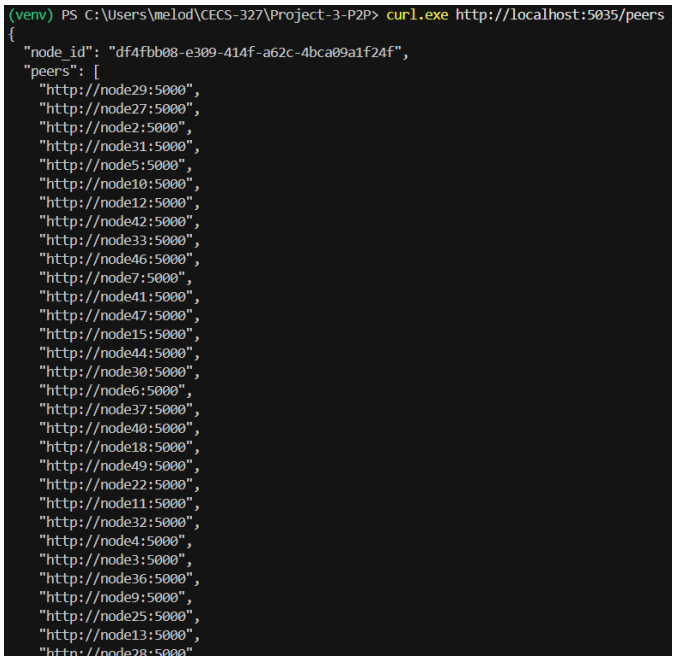
All peer operations use with **peers_lock**: for thread safety, preventing race conditions between the discovery thread and Flask request handlers.

Screenshots

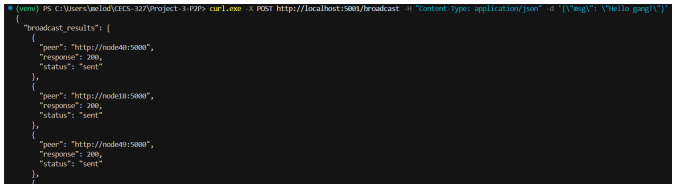
Bootstrap showing all registered peers:



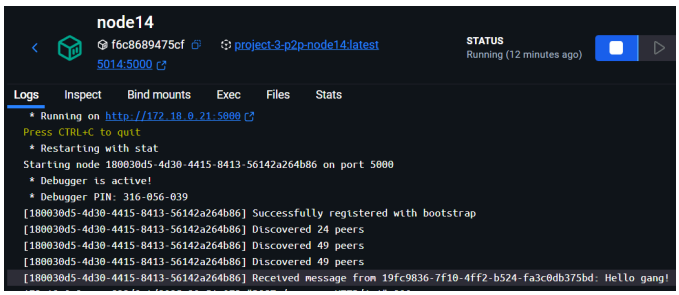
Individual nodes showing their peer lists:

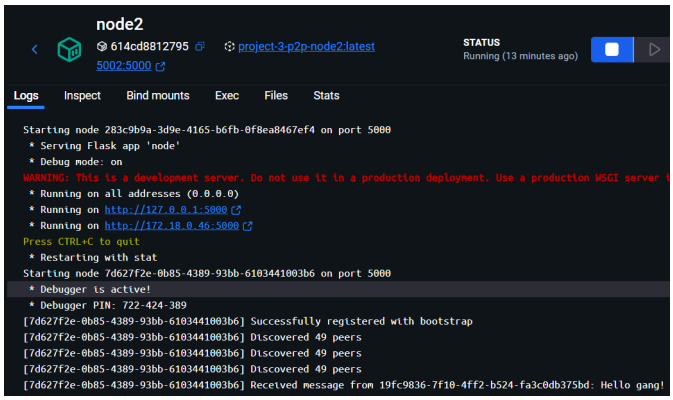


Successful broadcast response:



Logs showing received messages:

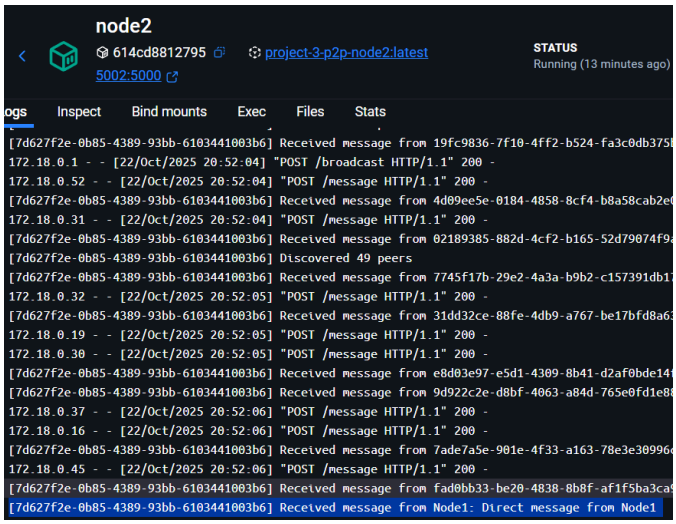




Test script output

```python
# Node 3 -> Node 1
test_pairs = [
    (active_nodes[0], active_nodes[1]),
    (active_nodes[1], active_nodes[2]),
    (active_nodes[2], active_nodes[0])
]

for sender, receiver in test_pairs:
    try:
        response = requests.post(
            f'http://localhost:{5000+receiver}/message',
            json={'sender': f'Node{sender}', 'msg': f'Direct message from Node{sender}'},
            timeout=5
        )
        if response.status_code == 200:
            print(f"   ✓ Node{sender} -> Node{receiver}: Message delivered")
        else:
            print(f"   X Node{sender} -> Node{receiver}: Failed")
    except Exception as e:
```



Challenges and Solutions:

- PowerShell JSON escaping: Used '{\"msg\": \"Hello\"}' format for curl.exe
- Container networking: Changed from localhost to Docker DNS names (http://node1:5000)
- Output buffering: Added flush=True to print statements for immediate logs
- Thread safety: Used threading.Lock() to prevent race conditions
- YAML typo: Fixed container-name to container_name in generate_compose.py

Individual Contributions:

Melody:

- Implemented all Python code (node.py, bootstrap.py, test_network.py, generate_compose.py)
- Tested 50-node deployment and captured screenshots
- Recorded demonstration video

Andrea:

- Created Dockerfiles and docker-compose.yml files
- Wrote README.md and project documentation

References:

Docker Documentation: https://docs.docker.com/
Flask Documentation: https://flask.palletsprojects.com/
Python Threading: https://docs.python.org/3/library/threading.html
CECS 327 Course Materials